Deterministic Mathematics for Technical AI Agents

An independently audited C++ feasibility demonstrator using reusable paceval execution graphs



Research question. Can a technical-agent tool call be validated as a defined engineering calculation, executed through a persistent mathematical engine and returned with correct units, explicit failures and reproducible numerical evidence?

Observed answer for the selected offline workload: Yes. Eight approved formula types were implemented using the original paceval C/C++ SDK and Linux runtime, with 100 new structured questions, 30 replayed requests and 20 intentionally invalid cases.

All 130 accepted outputs passed the independent 110-digit reference tolerance. The 20 invalid cases returned their expected rejection statuses. Twenty of 24 investigated TINC intervals contained the exact high-precision reference; four did not.

This report replaces the supplied November 2026 proposal with experimentally observed, scope-limited feasibility results. It is not a test of LLM reasoning, a production MCP server, a validated engineering design system or a performance comparison.

1 Research motivation and system boundary

Technical AI assistants may propose an expression, parameters and intended result unit. None of these elements should be assumed physically meaningful or safe solely because a language model supplied them. A useful mathematical tool must give a reproducible response for a precise declared calculation, and return a clear error for unsupported requests.

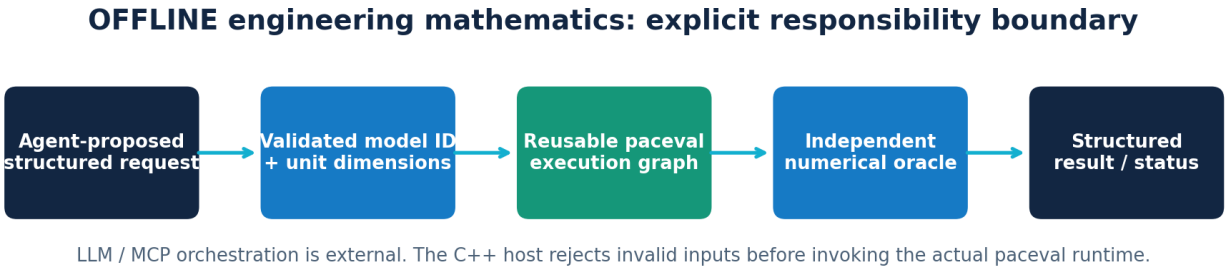


Figure 1. Implemented offline responsibility boundary. Only the mathematical expression engine is supplied by paceval; dimensional checks, policy and the independent reference are external.

The agent-facing contract separates five responsibilities: (1) structured request formatting; (2) allowlisted formula selection and unit validation; (3) persistent graph execution through paceval; (4) independent numerical checks; (5) a typed result with explicit rejection codes.

The approach follows the structure of MCP tool **inputSchema** and machine-readable outcomes. The provided JSONL adapter is a local integration demonstrator, not a server implementation, authentication or transport layer. Runtime input controls remain necessary in a future deployed endpoint. [2, 3]

Intentional security boundary: This prototype never evaluates arbitrary agent-supplied formula text. It compares the request against an audited registry of eight approved expressions, rejects unexpected syntax and does not expose filesystem, command execution or network operations through its computation tool.

Physical units and model selection remain external to paceval. Thus successful numerical output does not mean that an agent chose the correct physical model.

2 Eight implemented engineering models

Each model declares an exact paceval expression, unit dimensions for its inputs, domain restrictions and a named SI-compatible result. Six uniformly ordered variables (x0 ... x5) are declared for SDK consistency; unused input slots are empty in structured requests.

Formula registry ID	Expression	Output
axial_stress	x0/x1	Pa
electrical_power	x0*x1	W
kinetic_energy	0.5*x0*x1^2	J
heat_flow	x0*(x1-x2)	W
voltage_drop	x0*x1	V
cantilever_deflection	x0*x1^3/(3*x2*x3)	m
aerodynamic_drag	0.5*x0*x1*x2*x3^2	N
proposal_proxy	(x0*x1)/(x2+x3)+x4*sqrt(x5*x5)	N/m

Physics assumptions. The cantilever formula describes an end-loaded, small-deflection linear Euler-Bernoulli beam; the drag law is a simplified steady quadratic relationship; heat_flow uses a lumped conductance and absolute Kelvin temperatures. These are test models, not calibrated device or plant descriptions.

Connection to the submitted proposal. The unusual r = (F L)/(A + eps) + k sqrt(v v) expression is implemented exactly as a synthetic proposal_proxy where F is in newtons, L in metres, A and eps in square metres, v in metres/second, and k in N s/m2. Its two terms therefore both have dimension N/m. It is not asserted to be a recognized engineering law.

Source code: **src/technical_agent.cpp**. The allowlisted units, dimensions and domain checks are explicit; no general arbitrary-expression dimensional algebra is implemented.

3 Unit validation and tool-call contract

Unit conversion is performed before mathematical evaluation. The host represents units using five integer SI-base exponents (kg, m, s, A, K). The requested input dimensions must exactly match the registry entry and the output unit must be dimensionally compatible. Only documented multiplicative conversions are allowed.

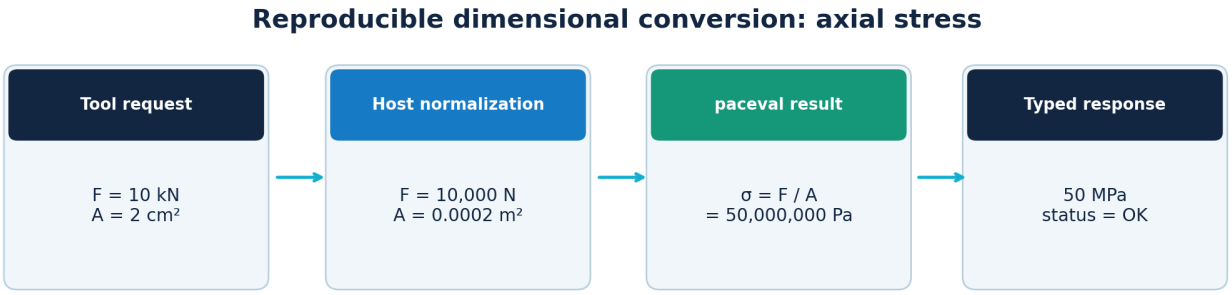


Figure 2. Verified example: 10 kN over 2 cm2 is 50 MPa. This operation was also executed through the JSONL adapter using the original paceval library.

A JSONL request contains model_id, approved expression, output_unit and six ordered {value, unit} slots. Strict key validation is performed in the Python adapter before writing a temporary casebook. C++ additionally checks the model, expression, dimensions, ranges, finiteness and numeric domain; it is authoritative for execution.

The **tool_schema.json** is a tool-description example compatible with a schema-based integration architecture. It does not automatically deploy a Model Context Protocol server. The sample local adapter runs one C++ batch per JSONL file and returns one structured result per accepted schema request.

The supported unit catalog is deliberately small and is **not a full UCUM implementation**. No offset temperatures (such as Celsius) or ambiguous units are silently accepted. For real engineering systems the unit policy needs review against applicable standards and operating envelopes.

See **examples/agent_calls.jsonl**, **examples/tool_schema.json**, and **src/agent_tool_adapter.py**.

4 Casebook: 150 executed requests

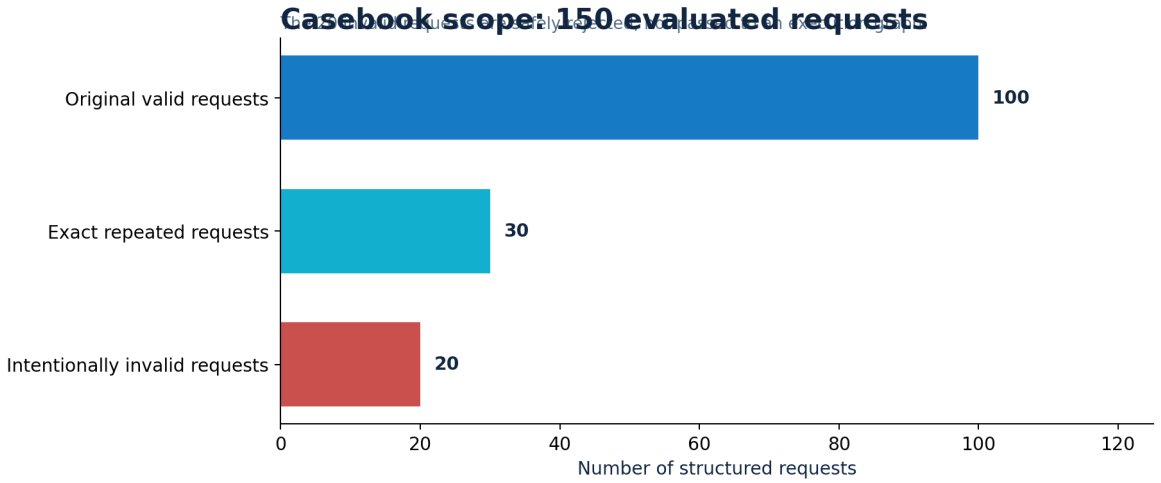


Figure 3. Casebook composition: 100 new structured engineering questions, 30 exact replays and 20 intentionally invalid requests.

The 100 original requests include converted values across the eight formula families. The 30 replays repeat previously accepted inputs with new request IDs and exercise persistent graph reuse. The 20 invalid requests cover unknown models, malformed or unauthorized equations, incompatible input/output units, nonfinite or missing numbers, and singular denominator cases.

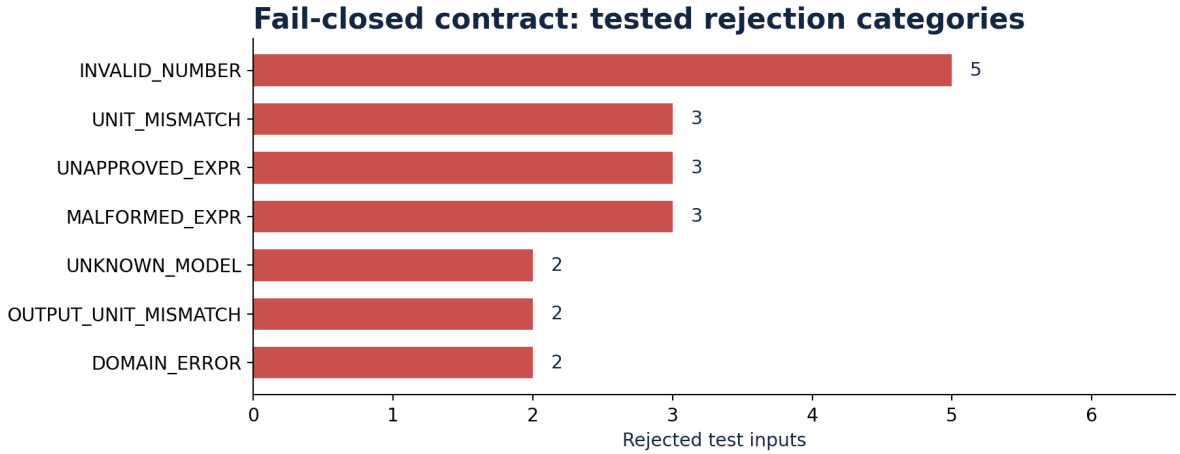


Figure 4. Every invalid request returned the expected rejection category; invalid data were rejected before paceval evaluation.

All 150 expected statuses matched. The casebook is synthetic and deterministically generated; it is not an externally annotated corpus of natural-language engineering questions.

Published data: **examples/requests.csv** and **examples/agent_results.csv**.

5 paceval implementation and numerical verification

The original SDK facade is compiled from **paceval_main.h** and **paceval_main.cpp**, with the vendor mathematical runtime linked as an external shared object. The application initializes the library, creates eight persistent normal graphs plus eight separately TINC-enabled graphs, evaluates each approved model across grouped requests using the documented batch API, and releases every handle before calling FreeLibrary. [1]

Key calls are **paceval_InitializeLibrary**, **paceval_CreateComputation**, **paceval_dGetComputationResultExt**, **paceval_dGetComputationResult**, **paceval_GetIsError**, **paceval_DeleteComputation**, and **paceval_FreeLibrary**.

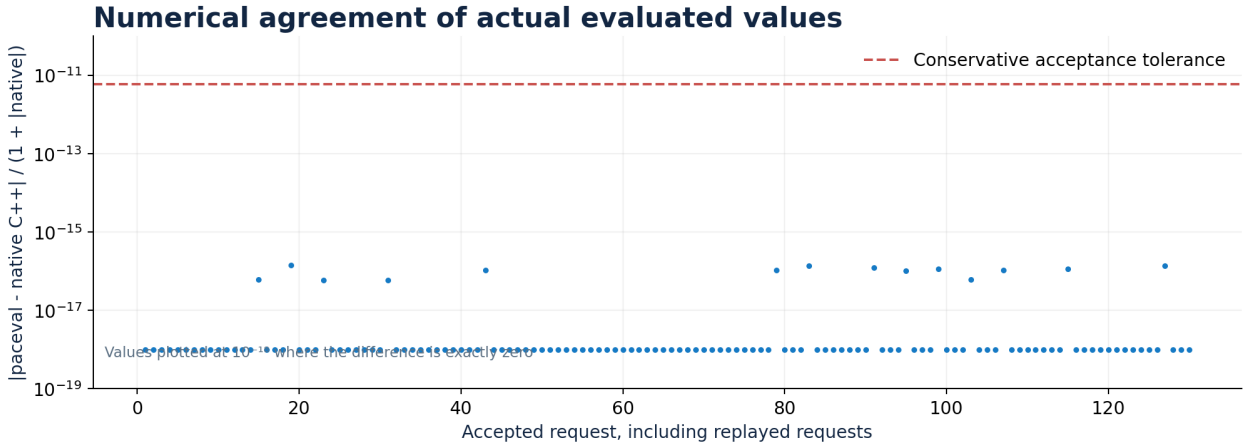


Figure 5. All 130 accepted results agree with independently implemented native C++ calculations within the test criterion. Each reported accepted output is also checked against an independent 110-decimal-digit mpmath computation.

The project uses the criterion |paceval - reference| <= 6 x 10^-12 (1 + |reference|) in the user-requested output unit. The maximum observed ordinary double-precision absolute difference to native C++ in the test batch was about 3.55 x 10^-15 in the units used by the requests. This is evidence only for the tested synthetic domain.

GCC 14.2 and Clang 17 built the project; automated tests passed with both. AddressSanitizer and UndefinedBehaviorSanitizer also completed their automated tests without a reported failure.

The numerical core is direct, original SDK-based execution. No REST or remote MCP service is used as a substitute for the C++ engine.

6 TINC: independent audit of point intervals

For three samples per model, separate TINC-enabled handles report a numeric result and lower/upper limits. An independent mpmath oracle evaluates the same mathematical formula at 110-digit working precision, treating each published binary64 input value as an exact initial value.

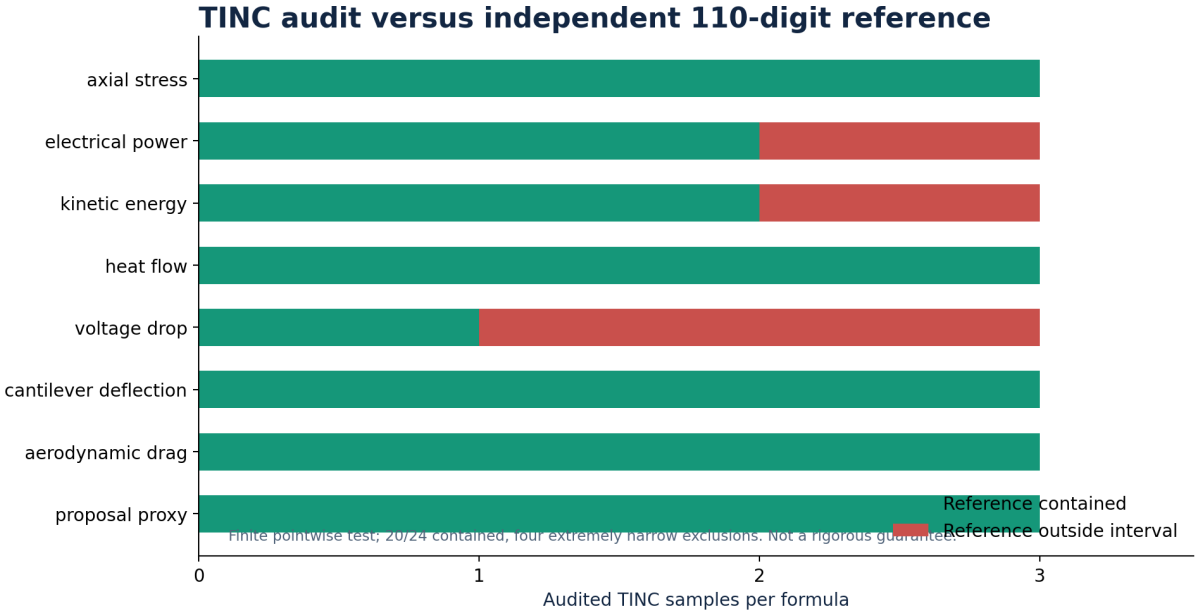


Figure 6. In 20 of 24 pointwise tests the exact reference was enclosed. Four results did not enclose it. This is not a benchmark of continuous interval uncertainty propagation.

Observed non-enclosures: Q018 (electrical power), Q011 (kinetic energy), Q013 and Q021 (voltage drop). These TINC intervals were extremely narrow or point-like and failed to enclose the exact real-number result of the published floating-point inputs.

Accordingly, this report makes **no universal or rigorous rounding-error enclosure claim**. A production implementation requiring certified bounds would need a separate reference with directed-rounding or rigorous ball arithmetic, appropriate continuous-domain tests and explicit failure treatment.

Source artifacts: **examples/tinc_samples.csv** and **src/high_precision_reference.py**. This limitation does not negate the independent finite-tolerance agreement of the ordinary computed values.

7 Reproducibility and research outlook

Environment. Linux x86-64; GCC 14.2 and Clang 17; C++17; original paceval Linux runtime with SHA-256 acf63a87df0ee822b467edbca887393d31ec0ea03562e1abf802d2512a5bfb12. The runtime binary is an external dependency and is deliberately excluded from the repository. No performance or energy measurements are reported.

Rebuild. From the repository root set an absolute PACEVAL_LIB path, run **cmake -S src -B build**, **cmake --build build -j**, then **ctest --test-dir build --output-on-failure**. Run **build/technical_agent** against **examples/requests.csv** and audit the resulting files with **src/high_precision_reference.py** (mpmath required). Full commands appear in README.md.

The local JSONL demonstration is run through **src/agent_tool_adapter.py**, using **examples/agent_calls.jsonl**. It correctly returns 50 MPa for the 10 kN / 2 cm2 query, 36 W for 24 V x 1.5 A, a unit-mismatch rejection and a schema-error rejection.

Future work. Connect a real agent through a safely authenticated tool server; test agent-generated unit hypotheses separately from arithmetic accuracy; allow richer expressions only after robust independent parsing and dimensional analysis; integrate a trusted units library; add human-reviewed model selection; assess larger verified formula catalogs and additional numerical reference backends.

The achieved research outcome is **feasible, deterministic and independently checked execution of approved engineering calculations through paceval**, with explicit machine-readable failures and demonstrated graph reuse. This does not establish automatic correctness of engineering assumptions, real-world decisions, or production readiness.

Selected references

[1] paceval UG, public C/C++ API; [2] Model Context Protocol, tools specification, 2025-11-25; [3] OpenAI, function calling documentation; [4] NIST SI resources.

Research scope derived from the supplied five-page proposal "Deterministic Mathematics for Technical AI Agents" (November 2026); implemented extensions and measured conclusions are identified in this report.